Author: Suhas Suresha & Dewang Sultania

SECTION 1

Tools as platform-managed capabilities (contract, definitions, SDK)

- **Platform inversion:** Tools are inventory objects with identity and lifecycle, not one-off Python functions copied into every workflow.
- **Four verbs:** Register, Discover, Execute, and Validate cover definition, catalog lookup, guarded invocation, and pre-flight argument checks.
- **Three definition layers:** Identity (name, version, owner) serves governance; schema feeds models; operational metadata drives policy, billing, and retries.
- **Behavior flags:** Fields such as read-only, idempotent, rate limits, and side-effect notes tell the platform how aggressively to retry or confirm.
- **SDK simplicity:** Sarah registers once, discovers namespaces, passes schemas into `chat`, and calls `platform.tools.execute` while secrets stay server-side.

SECTION 2

Registry, namespacing, discovery, versions

- **Hierarchical names:** Dotted names like `healthcare.scheduling.book_appointment` prevent collisions when many teams ship similarly labeled functions.
- **Permissioned discovery:** Namespace filters and auth context ensure applications only see tools they may invoke, not the entire catalog.
- **Capability search:** Tags and capability lists let agents find "insurance verification" without hardcoding every integration path.
- **Semantic versioning:** Multiple semver releases coexist; callers pin version constraints while the registry rejects incompatible minor bumps.
- **Deprecation path:** Sunset dates, migration guides, and diff tooling let tools evolve without silent schema breaks in production prompts.

SECTION 3

Execution (adapters, credential isolation, sync/async)

- **Adapters:** Vendor complexity—OAuth, FHIR shapes, nested JSON—lives in adapter classes that expose a stable platform result type.
- **Credential references:** Registration stores `credential_ref` names only; the Credential Store retrieves, scopes, rotates, and injects values at execute time.
- **Sync vs. async:** Fast reads block inside a timeout wall; long insurance or authorization flows return task ids with poll or webhook completion.
- **Execution limits:** Per-tool timeouts, memory caps, response size ceilings, and retry budgets stop runaway calls from starving the fleet.
- **Idempotency discipline:** Retries are safe only when metadata or adapter keys guarantee at-most-once side effects for mutating calls.

SECTION 4

MCP interoperability and what MCP leaves to the platform

- **Integration arithmetic:** Custom N×M glue collapses toward N+M with MCP, and a central Tool Service can reduce organizational cost toward M shared server connections.
- **Primitives:** MCP tools, read-only resources, and reusable prompts map differently; the Tool Service primarily owns executable tools.
- **Standard wire only:** MCP defines listing and invocation transport—not secret rotation, org rate limits, namespace ACLs, or cost attribution.
- **register_mcp_server:** External servers register under a platform namespace, pull credentials from the store, and accept policy overrides like confirmation gates.
- **Complementary layers:** MCP and the platform are additive: protocol for interoperability, platform for enterprise controls and audit.

SECTION 5

Execution safeguards

- **Per-invocation isolation:** Resource limits mirror workflow sandboxing at finer granularity so a toxic adapter cannot exhaust shared workers.
- **Circuit breakers:** Per-tool closed → open → half-open state machines fail fast when vendors are unhealthy instead of retry-storming them.
- **Structured errors:** Timeouts and breaker trips return model-readable failures rather than raw stack traces or credential leaks.
- **Operational visibility:** Breaker transitions and limit hits belong in metrics so on-call engineers see vendor pain before users flood support.

SECTION 6

Guardrails (input, output, behavioral) and declarative policy

- **Beyond profanity filters:** Execution policies block bookings without referrals, enforce referral rules, and encode domain logic generic safety APIs miss.
- **Five inspection points:** Checks span user input, proposed tools, arguments, external results, and final output for defense in depth.
- **Input and output:** Injection classifiers, topic scope, PII handling, grounding checks, and tiered block/transform actions protect users and the org.
- **Behavioral rules:** Session permissions, cross-tool consistency, per-session rate caps, and human-in-the-loop gates pause high-stakes mutations.
- **Declarative YAML:** Compliance edits versioned policy documents hot-loaded by the Guardrails Service; workflows catch `GuardrailViolationError` with user-safe text.
- **Observability:** Shadow mode, canary rollout, structured audit rows, and metrics on allow/deny rates make safety decisions measurable rather than mysterious.